

Import (CodeGrade)



<https://github.com/learn-co-curriculum/python-p3-import>



<https://github.com/learn-co-curriculum/python-p3-import/issues/new>

Learning Goals

- Importing modules using absolute and relative imports.
-

Key Vocab

- **Module**: a file containing Python definitions and statements. A module's functions, classes, and global variables can be accessed by other modules.
 - **Package**: a collection of modules that can be accessed as a group using the package name.
 - **import** : the Python keyword used to access data from other packages and modules inside of the current module.
 - **PyPI**: the **Python Package Index**. A repository of Python packages that can be downloaded and made available to your application.
 - **pip** : the command line tool used to download packages from PyPI. **pip** is installed on your computer automatically when you download Python.
 - **Virtual Environment**: a collection of modules, packages, and scripts that can be activated or deactivated at any time.
 - **Pipenv**: a virtual environment tool that uses **pip** to manage the modules, packages, and scripts that you intend to use in your application.
-

Introduction

As your programs get longer, you may want to split them into several files for easier maintenance. You may also want to use a handy function that you've written in several programs without copying its definition into each program, or functions that others wrote to save you a bit of

development time. All imports start from files (also called **modules**). Using the **import** keyword, we can access whole modules or the classes, functions, and objects inside from any other module.

Run `pipenv install` and `pipenv shell` to enter the virtual environment.

Importing a Module

Lets use the Python `os` module as an example. We will use the `name` property to demonstrate the different ways to import.

Open the Python shell and enter the following code:

```
$ python
>>> import os
>>> os.name
# => 'posix'
```

"posix", or the Portable Operating System Interface, is the name of the module that code is executed from in the Python shell.

A module may have multiple functions and definitions. What if we only want to use a few of them? A more concise way of importing is only importing the things we need. Lets say we want to import only the `name` property in the `os` module. We can do this by using the `from` keyword.

```
$ python
>>> from os import name
>>> name
# => 'posix'
```

Note how we did not need to prepend `os` to the `name` .

The `from` keyword also allows us to import everything from a library using the `*` operator.

```
$ python
>>> from os import *
>>> name
# => 'posix'
```

This can be bad practice, because it is less concise and you may run into namespace issues. (What if we had called another variable `name` ?) When using the `*` operator we don't know exactly what we are taking from the module, leading to less readable code.

Absolute Imports

An absolute import specifies the module to be imported using the full path from the project root directory.

```
absolute_package
├─ package1
│   ├─ module1.py
│   └─ module2.py
├─ package2
│   ├─ module3.py
│   ├─ module4.py
│   ├─ module5.py
│   └─ subpackage1
│       └─ module6.py
```

Lets use the above file structure to practice. All the code below will be run in the Python REPL. Change directory into the `absolute_package/` directory. To activate the REPL run `python` in the terminal.

Let's assume that there is a function called `function1()` in each module:

```
$ python
>>> from package1 import module1
```

```
>>> module1.function1()  
# => Function 1 in module 1
```

We can explicitly import this function as follows:

```
$ python  
>>> from package1.module1 import function1  
>>> function1()  
# => Function 1 in module 1
```

`module6` is buried a bit deeper- to import its `function1()` , we need to crawl through `subpackage1/` in `package2` :

```
$ python  
>>> from package2.subpackage1.module6 import function1  
>>> function1()  
# => Function 1 in module 6
```

Pros and Cons of Absolute Imports

Absolute imports remain valid even if the current location of the import statement changes and they make it easy to see exactly where the resource is coming from.

When Should We Not Use Absolute Imports?

Lets say we have want to import a module from a long nested directory structure. The import statement would be huge:

```
from package1.subpackage1.subpackage2.subpackage3.module1 import function1
```

We can avoid this by using **relative imports**.

Relative Imports

The reason why relative imports exist is because they allow us to rearrange the structure of large packages without having to edit sub-packages. If we rearrange a large package with absolute imports, we have to change all the paths in every file's import statements to match the new location.

Lets use the following directory structure:

```
relative_package
├── __init__.py
├── subpackage1
│   ├── subpackage2
│   │   ├── __init__.py
│   │   ├── module1.py
│   │   └── module2.py
└── module3.py
```

Lets say `module1` and `module2` have the following code:

```
# module 1
def function1():
    print('Function 1 from module1')
```

```
# module 2
from .module1 import function1
```

```
function1()
```

Now you can change directory to the parent directory of `subpackage1` . We can run `module2` as a module using the following command.

```
$ python -m subpackage1.subpackage2.module2
```

```
# => Function 1 from module1
```

Relative Imports only work when we use them in a module. Which is why we need to include the `-m` argument. Relative imports cannot be invoked in scripts.

If you need to go up multiple levels in the directory structure you can use additional dots.

In module2 we can import module3 using two dots which represents the parent directory that module3 is in. In this example we import the entire module not just one function like the last example.

```
from .module1 import function1
from .. import module3
```

```
function1()
module3.function1()
```

```
$ python -m subpackage1.subpackage2.module2
# => Function 1 from module1
# => Function 1 from module3
```

Relative imports are great if you have lots of code files that are related, but it can be unclear to other developers which modules are kept where. PEP-8 suggests that you only use relative imports when the absolute path extends past 79 characters, but it's a good tool to have at your disposal.

Conclusion

We can use import statements to pull code from various different sources like the Python standard libraries, other modules within our project and Python packages we pull using the `pip` command which we will learn more about in the following lessons.

Resources

- [Python 3 Documentation](https://docs.python.org/3/)  (https://docs.python.org/3/)
- [Python import system](https://docs.python.org/3/reference/import.html)  (https://docs.python.org/3/reference/import.html)

This tool needs to be loaded in a new browser window

Load Import (CodeGrade) in a new window